# Experiment 3: Design of Test Cases and a Maintenance Plan for a Sample Software Module

## Aim

To design test cases for a sample software module and prepare a maintenance plan for it.

## Objective

- Understand the purpose of software testing.
- Design positive and negative test cases for a module.
- Study the types of software maintenance.
- Prepare a maintenance plan for the module.

## Selected Module

**User Login Module** of a web application. The user enters a username and password; the system validates them and opens the dashboard. After 3 wrong attempts the account is locked.

## Test Design Techniques Used

- **Equivalence partitioning** - inputs divided into valid and invalid groups.
- **Boundary value analysis** - password length checked at minimum (8) and maximum (20) limits.

## Test Cases

| ID | Scenario | Input | Expected Result | Status |
|---|---|---|---|---|
| TC01 | Valid login | Correct username and password | Dashboard opens | Pass |
| TC02 | Wrong password | Valid username, wrong password | Error: Invalid credentials | Pass |
| TC03 | Wrong username | Unknown username, any password | Error: Invalid credentials | Pass |
| TC04 | Empty fields | Both fields blank | Error: Fields are required | Pass |
| TC05 | Password too short | 7 characters | Validation message shown | Pass |
| TC06 | Minimum length | 8 characters (valid) | Login accepted | Pass |
| TC07 | Maximum length | 20 characters (valid) | Login accepted | Pass |
| TC08 | Account lock | 3 wrong attempts in a row | Account locked for 15 minutes | Pass |
| TC09 | SQL injection | ' OR '1'='1 in username | Input rejected, no login | Pass |
| TC10 | Session timeout | Idle for 15 minutes | User logged out automatically | Pass |

*Note: the Status column should be updated with your actual test run results.*

## Software Maintenance Plan

### Types of maintenance

| Type | Purpose | Example for Login Module |
|---|---|---|
| Corrective | Fix defects found after release | Fix error when username contains capital letters |
| Adaptive | Adjust to a new environment | Support a new browser version or OS |
| Perfective | Improve features and performance | Add 'Forgot password' and OTP login |
| Preventive | Avoid future problems | Refactor code, update libraries, add logging |

### Maintenance schedule

| Activity | Frequency | Responsible |
|---|---|---|
| Bug fix release | As reported (within 3 working days) | Developer |
| Security patch and library update | Monthly | Developer |
| Regression testing | After every change | Tester |
| Performance and log review | Quarterly | Team lead |
| Backup verification | Weekly | System admin |

## Results

Ten test cases covering valid, invalid, boundary and security scenarios were designed for the login module, and a maintenance plan with all four maintenance types and a schedule was prepared.

## Conclusion

Well-designed test cases find defects early, and a planned maintenance schedule keeps the software secure and useful after release. Testing and maintenance together make up a large share of the total software life cycle cost.